DS 501 · INTRODUCTION TO DATA SCIENCE

Week 04 · Databases and SQL

Reading Summaries

Two slides per paper: the problem and key ideas, then what to take away and how it connects to this week. Use them after your own first pass, not instead of it.

THE PAPERS

R1 **Codd 1970**
Required · Relations Protect Users From Storage Details

R2 **Cattell 2010**
Required · Comparing Scalable SQL and NoSQL Data Stores

01 **Brewer 2012**
Optional · CAP Is a Trade-off to Manage, Not Pick Two of Three

02 **Stonebraker & Pavlo 2024**
Optional · SQL and the Relational Model Keep Absorbing Rivals

International Balkan University

Required

Relations Protect Users From Storage Details

Codd, E. F. (1970). A relational model of data for large shared data banks. Communications of the ACM, 13(6), 377–387. https://doi.org/10.1145/362384.362685

THE PROBLEM

In 1970, programs depended on how data was stored: record order, index chains, and tree or network access paths. When storage changed, programs broke. Codd asks how to shield users and programs from these internal details.

KEY IDEAS

01 Describe all data as relations: sets of rows with no meaningful row order and no duplicate rows

02 Primary keys identify rows; foreign keys refer to other relations by value, not by pointers

03 Normalization turns nested repeating groups into flat tables that carry the parent's key

Key term: Data independence — Programs and users keep working when the storage order, indexes or access paths of the data change

Required

Say What Data You Want, Not How to Reach It

WHAT TO TAKE AWAY

1. Names three dependencies to remove: on stored ordering, on indexes, and on access paths
2. Proposes a high-level data language based on predicate logic, the idea that later led to SQL
3. Defines projection, join and restriction, and uses them to reason about redundancy and consistency

THIS WEEK

Chinook's tables (Artist, Album, Track, Invoice) follow this model: keys link rows, and your SQL from Python never mentions file layout or indexes. This is the core of lesson 4-1.

LIMITATION

The paper is theory: it does not cover implementation or a full query language. SQL later departed from it, e.g. by allowing duplicate rows.

For your half-page note: Take one Chinook query. Which storage details (row order, indexes, file layout) does it not depend on, and why does that help?

Required

Comparing Scalable SQL and NoSQL Data Stores

Cattell, R. (2010). Scalable SQL and NoSQL data stores. ACM SIGMOD Record, 39(4), 12–27. https://doi.org/10.1145/1978915.1978919

THE PROBLEM

Web applications with millions of users need many small reads and writes per second. Traditional databases scaled poorly across many servers, and dozens of new data stores appeared. Cattell surveys and compares over twenty of them.

KEY IDEAS

01 Groups stores by data model: key-value, document, extensible record (wide column), scalable relational

02 NoSQL stores scale out by partitioning (sharding) and replicating data over many servers

03 Most give up full ACID transactions for weaker, often eventual, consistency

Key term: Horizontal scaling — Spreading data and load over many servers that share no RAM or disk, instead of using one bigger machine

Required

No Single Data Store Is Best for Every Use

WHAT TO TAKE AWAY

1. Key-value stores fit lookups by one key; document stores fit varied objects searched on many fields
2. Scalable SQL systems can compete if operations and transactions mostly stay on one node
3. Predicts NoSQL will last, new relational systems will grow, and products will consolidate

THIS WEEK

Gives lesson 4-3 its map of NoSQL store types. Compare the joins and transactions you use on Chinook with stores that offer only key lookups or single-record updates.

LIMITATION

A 2010 snapshot: many systems have since changed or disappeared, and the author notes that few independent benchmarks backed the scaling claims.

For your half-page note: For the Chinook music store, which data would you keep relational, and which might fit a key-value or document store? Why?

Optional

CAP Is a Trade-off to Manage, Not Pick Two of Three

Brewer, E. (2012). CAP twelve years later: How the "rules" have changed. Computer, 45(2), 23–29. https://doi.org/10.1109/MC.2012.37

THE PROBLEM

The CAP theorem was often read as: choose two of consistency, availability and partition tolerance. Brewer, who first proposed CAP, argues this reading is misleading and explains how designers should handle network partitions.

KEY IDEAS

01 Partitions are rare, so without one a system need not give up consistency or availability

02 C and A are not on/off; the choice can differ per operation, per data item, or per subsystem

03 Handle partitions in three steps: detect the partition, enter partition mode, then recover

Key term: Network partition — A communication failure that splits a distributed system into groups of nodes that cannot reach each other

Optional

Plan How to Recover From Partitions

WHAT TO TAKE AWAY

1. A partition is a time limit on communication: on timeout, cancel (less A) or go on (risk to C)
2. During a partition, delay or limit risky operations, e.g. cap ATM withdrawals, and compensate later
3. Version vectors and CRDTs help replicas merge their state after a partition ends

THIS WEEK

Explains the consistency trade-offs behind the NoSQL stores in lesson 4-3. Our Chinook lab runs SQLite on one machine, so it never faces this choice.

LIMITATION

An essay with examples, not a measured study: it gives design advice but no formal proofs or performance results for its techniques.

To think about: An online music store's data centres lose contact. Should it keep selling? Which rule could break, and how would you fix it later?

Optional

SQL and the Relational Model Keep Absorbing Rivals

Stonebraker, M., & Pavlo, A. (2024). What goes around comes around... and around... ACM SIGMOD Record, 53(2), 21–37. https://doi.org/10.1145/3685980.3685984

THE PROBLEM

Since 2005, many systems tried to replace SQL or the relational model: MapReduce, key-value, document, column-family, text, array, vector and graph systems. The authors ask what happened to them over 20 years.

KEY IDEAS

01 Most non-relational systems stayed niche or are moving toward SQL and the relational model

02 Almost every major NoSQL system added a SQL-like language, and many added ACID transactions

03 The big changes were in how relational systems are built: column stores, cloud, data lakes

Key term: Lakehouse — An engine that runs SQL and DataFrame work over open-format files, such as Parquet, kept in cloud object storage

Optional

New Data Models Rarely Replace SQL; SQL Absorbs Them

WHAT TO TAKE AWAY

1. MapReduce and Hadoop faded; document stores and relational systems are converging
2. Relational systems added vector search fast, and SQL:2023 added property graph queries
3. Column storage now dominates analytics, and the cloud has reshaped how databases are built

THIS WEEK

Updates Cattell's 2010 picture for lesson 4-3 and supports lesson 4-2: SQL skills from the Chinook lab carry over to many modern systems. It also discusses pandas DataFrames.

LIMITATION

A strongly argued opinion paper by two relational-database researchers. Its predictions are judgements, not measured results.

To think about: Cattell (2010) predicted NoSQL would last. What do Stonebraker and Pavlo say happened by 2024, and do you agree?

Writing Your Half-Page Note

01

The problem

What was broken or missing before this paper? One or two sentences.

02

The key idea

The one idea you would keep if you forgot everything else. Your own words.

03

The connection

Where it shows up this week: a lesson, the lab, or your project data.

Half a page means about 150–200 words. Bring it to the next class; one quiz question may refer to the required reading.